

Secure Boot e Secure Update

Come, quando e perché implementarli su ESP32

Guida introduttiva per sviluppatori firmware embedded

Preparato per: Samuel — Carlo Gavazzi Controls, R&D

Data: 2 settembre 2026

Ambito: ESP32 / ESP-IDF, prodotti di energy metering connessi

Livello: introduttivo, con parte visiva per la memorizzazione

Nota: i riferimenti normativi e i link ai documenti Espressif sono riportati a memoria e non sono stati verificati online al momento della stesura. Sono segnalati nel testo come da verificare prima di un uso formale (documenti di conformità, procedure di produzione).

Indice

Indice	2
1. Come leggere questo documento	4
2. Il problema che stiamo risolvendo	6
3. Fondamenti: quel poco di crittografia che serve	7
3.1 Hash: l'impronta digitale di un file	7
3.2 Firma digitale: l'impronta che solo tu puoi produrre	7
3.3 Cifratura: rendere illeggibile	8
3.4 Memoria OTP: dove vive la fiducia	8
4. Secure Boot	10
4.1 In una frase	10
4.2 Perché implementarlo	10
4.3 Quando implementarlo	10
4.4 Come funziona su ESP32: Secure Boot V2	12
4.5 Come abilitarlo: procedura passo-passo	15
4.6 Flash Encryption: il complemento naturale	16
5. Secure Update	17
5.1 In una frase	17
5.2 Perché implementarlo	17
5.3 Quando implementarlo	17
5.4 Come funziona su ESP32: il flusso OTA	19
5.5 Rollback: stati dell'applicazione	21
5.6 Anti-rollback: la versione di sicurezza	21
5.7 Verifica di firma senza Secure Boot completo	23
5.8 Il canale: TLS e pinning	23
5.9 Cifratura del pacchetto OTA	23
6. Gestione delle chiavi di firma	24
6.1 Principi	24
6.2 Rotazione e revoca	25
6.3 Cosa NON fare	25
7. Riferimenti normativi: CRA e RED-DA	26
7.1 Cyber Resilience Act — Reg. (UE) 2024/2847	26
7.2 RED Delegated Act — Reg. delegato (UE) 2022/30	27
7.3 Come usarli nel progetto	27

8. Checklist implementativa	28
8.1 Progettazione	28
8.2 Chiavi e infrastruttura	28
8.3 Firmware	28
8.4 Produzione e validazione	28
9. Errori comuni e come riconoscerli	30
10. Glossario	31
11. Riferimenti	33
11.1 Documentazione Espressif (ESP-IDF)	33
11.2 Normativa e standard	33
11.3 Prossimi passi suggeriti	33

1. Come leggere questo documento

Questo documento risponde a tre domande su due meccanismi: **che cosa** sono Secure Boot e Secure Update, **perché** e **quando** implementarli, e **come** farlo concretamente su ESP32 con ESP-IDF. È pensato per chi ha già esperienza di firmware embedded ma non ha ancora lavorato su un progetto con firmware firmato: si parte dai fondamenti crittografici strettamente necessari (senza matematica) e si arriva ai comandi e alle opzioni di configurazione.

Ogni sezione tecnica ha uno **schema** con una spiegazione scritta. Gli schemi sono costruiti per essere ricordati: stessi colori in tutto il documento, con questa convenzione:

Colore	Significato
Rosso	Radice di fiducia, chiave privata, minacce, punti irreversibili o pericolosi
Arancione	Bootloader di secondo stadio, stati intermedi, decisioni, avvertenze
Verde	Applicazione verificata, chiave pubblica, esito positivo, mitigazioni
Blu	Memoria non volatile (eFuse, otadata), infrastruttura, informazioni

Se hai poco tempo: leggi la mappa mentale qui sotto, il capitolo 3 (fondamenti, [§ 3](#)) e le due sezioni «In una frase» che aprono i capitoli 4 e 5 ([§ 4.1](#), [§ 5.1](#)). Il resto è approfondimento e materiale operativo per quando si passa all'implementazione.

Rimandi interni

Molti concetti vengono nominati prima di essere spiegati a fondo. Dove succede, trovi un rimando del tipo → [§ 4.4](#): è un collegamento cliccabile alla sezione in cui quel concetto è trattato in modo completo. Alla prima lettura puoi ignorarli e proseguire; servono quando un termine non è chiaro e vuoi andare subito alla spiegazione.

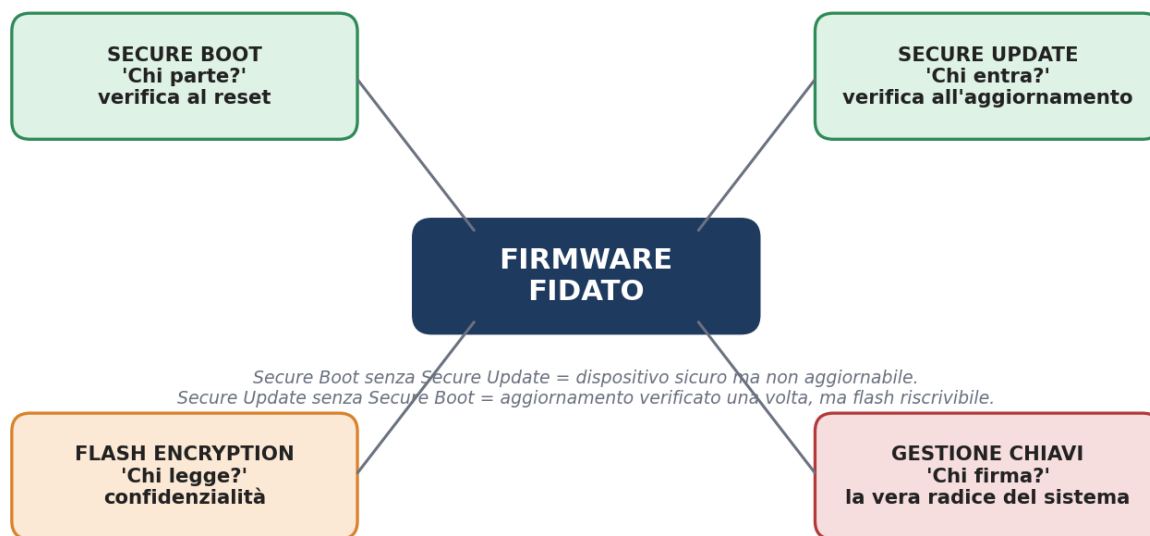


Figura 1 — Mappa mentale. Quattro meccanismi che ruotano intorno a un unico obiettivo: eseguire solo firmware fidato per tutta la vita del prodotto. Ogni riquadro risponde a una domanda diversa.

Da ricordare

Secure Boot risponde a «chi può **partire?**» (§ 4). Secure Update risponde a «chi può **entrare?**» (§ 5). Flash Encryption risponde a «chi può **leggere?**» (§ 4.6). La gestione delle chiavi decide «chi può **firmare?**» (§ 6) — ed è il punto da cui dipende tutto il resto.

2. Il problema che stiamo risolvendo

Un energy meter connesso è un dispositivo che resta installato per 10-15 anni in un quadro elettrico, spesso in luoghi accessibili a terzi, collegato a una rete e in grado di influenzare misure fiscali o decisioni di controllo. Il suo firmware è quindi un bersaglio interessante per tre categorie di attaccante:

- **Chi ha accesso fisico** al dispositivo (installatore, cliente, ladro di un campione): può collegare un programmatore alla flash SPI, leggerla, modificarla, riscriverla. Su un ESP32 senza protezioni bastano un cavo USB e `esptool.py`. → § 4
- **Chi ha accesso di rete**: può fingersi il server di aggiornamento, intercettare un OTA, inviare un firmware modificato o una versione vecchia con una vulnerabilità nota (downgrade attack). → § 5, § 5.6
- **Chi vuole il vostro codice**: estrarre il firmware permette di copiare algoritmi di misura, trovare credenziali hardcoded, studiare il protocollo per attaccare altri dispositivi. → § 4.6

Senza contromisure, il firmware in flash è un file come un altro: chiunque possa scriverlo può far eseguire al microcontrollore quello che vuole. Le contromisure di questo documento hanno un obiettivo preciso: fare in modo che **solo il firmware autorizzato dal produttore possa essere eseguito**, al primo avvio e dopo ogni aggiornamento.

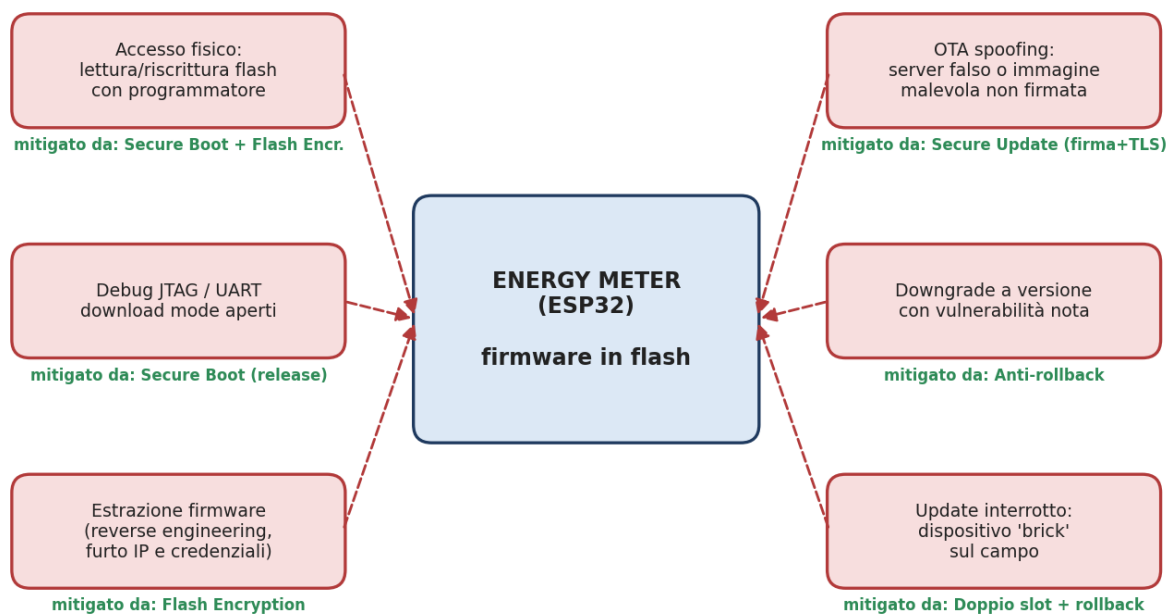


Figura 2 — Modello delle minacce semplificato per un dispositivo ESP32 sul campo. A sinistra le minacce che richiedono accesso fisico, a destra quelle legate all'aggiornamento remoto. Sotto ogni minaccia, il meccanismo che la mitiga.

La figura 2 vale come tabella di consultazione: ogni volta che si discute se «serve davvero» una funzione di sicurezza, si può risalire alla minaccia che essa neutralizza e chiedersi se quella minaccia è realistica per il prodotto. Per un dispositivo connesso di classe industriale, nell'Unione Europea dal 2027, la risposta è quasi sempre sì (vedi § 7).

3. Fondamenti: quel poco di crittografia che serve

Non è necessario capire RSA o le curve ellittiche per implementare Secure Boot. Servono tre concetti e la capacità di distinguerli tra loro, perché la confusione tra «integrità», «autenticità» e «confidenzialità» è la fonte più comune di scelte di progetto sbagliate.

3.1 Hash: l'impronta digitale di un file

Una funzione di hash (useremo sempre SHA-256) trasforma un file di qualsiasi lunghezza in una stringa fissa di 32 byte. Ha due proprietà che ci interessano: cambiando anche un solo bit del file, l'hash cambia completamente; e non è praticabile costruire un file diverso che produca lo stesso hash. Un hash dice quindi «questo file è esattamente quello di cui ho l'impronta» — garantisce l'**integrità**. Non dice nulla su *chi* lo ha prodotto: chiunque può calcolare l'hash di un firmware malevolo e allegarlo. Per questo serve la firma, → § 3.2.

3.2 Firma digitale: l'impronta che solo tu puoi produrre

La firma digitale usa due chiavi legate tra loro: una **privata**, che resta in fabbrica, e una **pubblica**, che viene messa nel dispositivo. Il meccanismo funziona in tre passi.

- **In fabbrica** si calcola l'impronta del firmware e la si «sigilla» con la chiave privata. Il risultato è la firma. Firmware e firma vengono spediti insieme.
- **Nel dispositivo** il bootloader rifà l'impronta del firmware che ha davanti e, con la chiave pubblica, apre il sigillo per leggere l'impronta calcolata in fabbrica. Se le due impronte coincidono, il firmware è identico a quello firmato e lo ha firmato chi possiede la chiave privata: questa è l'**autenticità**. Se non coincidono, il firmware non parte.
- La chiave pubblica serve **solo a controllare**, non a firmare. Quindi non è un segreto: può leggerla chiunque smonti il dispositivo, senza che questo lo renda meno sicuro. L'unica cosa da proteggere è la chiave privata, → § 6.

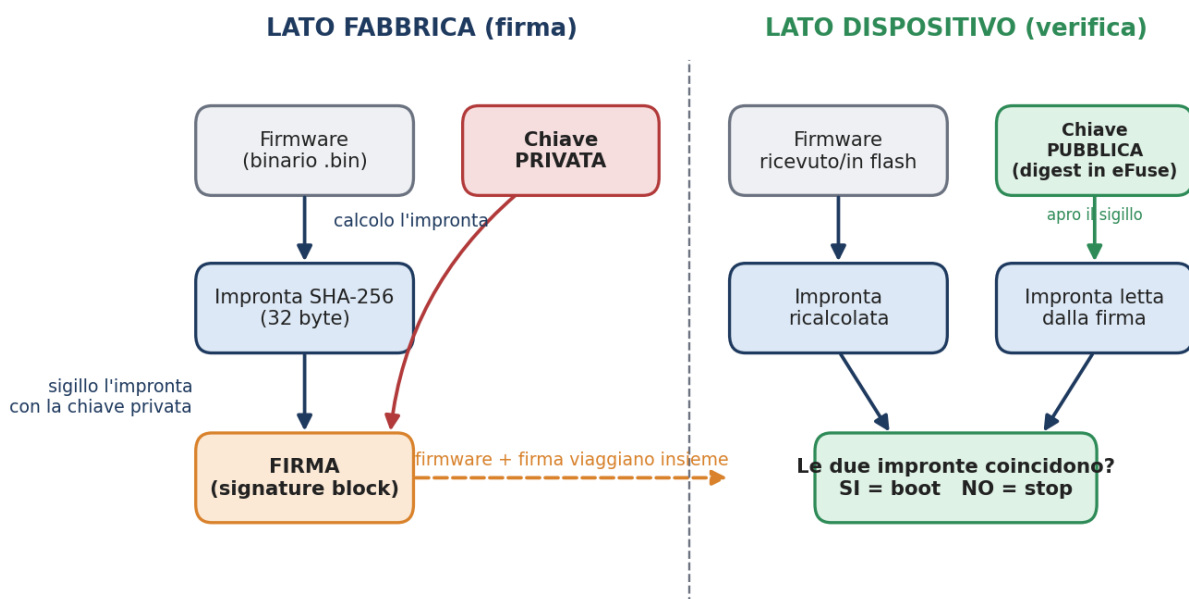


Figura 3 — Firma (a sinistra) e verifica (a destra). La chiave privata non lascia mai la fabbrica; nel dispositivo c'è solo la chiave pubblica. Firmware e firma viaggiano insieme.

Un'analogia

La chiave privata è il timbro dell'azienda; la chiave pubblica è il campione del timbro appeso alla parete dell'ufficio postale. Chiunque può guardare il campione e verificare se un timbro è autentico, ma solo chi ha il timbro può timbrare.

Il dettaglio ESP32 (si può saltare alla prima lettura)

Negli eFuse dell'ESP32 non c'è la chiave pubblica intera, ma solo la sua impronta SHA-256 (32 byte), perché lo spazio OTP è poco. La chiave pubblica completa viaggia dentro il signature block, appeso al firmware (struttura e posizione in § 4.4). Il bootloader fa quindi due controlli in sequenza: prima calcola l'impronta della chiave pubblica trovata nel signature block e verifica che coincida con quella in eFuse («è davvero la nostra chiave?»); solo dopo usa quella chiave per verificare la firma del firmware.

3.3 Cifratura: rendere illeggibile

La cifratura (AES) rende un dato illeggibile a chi non ha la chiave. È un meccanismo **indipendente** dalla firma: un firmware può essere firmato ma leggibile (chiunque può estrarlo e studiarlo, ma non modificarlo senza che il boot si fermi) oppure cifrato ma non firmato (nessuno lo legge, ma se qualcuno riscrive la flash con dati casuali cifrati il risultato è imprevedibile). Su ESP32 la cifratura della flash si chiama **Flash Encryption** e si abilita separatamente dal Secure Boot; nei prodotti di produzione si abilitano quasi sempre entrambi. → § 4.6

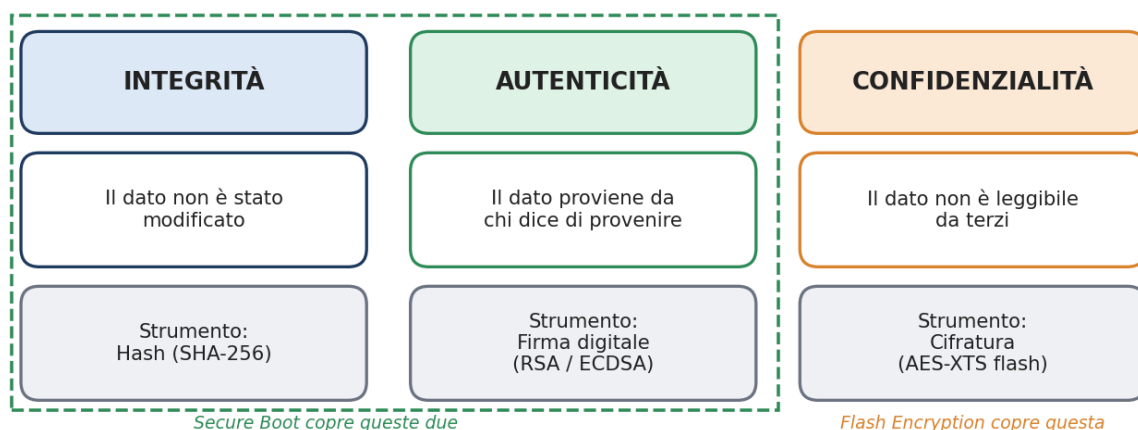


Figura 4 — Le tre proprietà e lo strumento che le garantisce. Secure Boot e Secure Update coprono integrità e autenticità; la confidenzialità del codice richiede Flash Encryption, che è un meccanismo separato.

Errore classico

«Il firmware è cifrato, quindi è sicuro.» No: la cifratura non impedisce a un attaccante con la chiave di cifratura (o con un bug che gliela fa ottenere) di scrivere codice arbitrario, e non impedisce il downgrade. Solo la firma verificata a ogni boot garantisce che il codice eseguito sia quello autorizzato.

3.4 Memoria OTP: dove vive la fiducia

Tutto il meccanismo poggia su un dato che l'attaccante non deve poter cambiare: il digest della chiave pubblica. Se fosse in flash, basterebbe riscriverlo insieme al firmware malevolo. Per questo si usano gli **eFuse**: bit programmabili una sola volta (one-time programmable, OTP) all'interno del

chip. Una volta «bruciato» un eFuse a 1 non torna a 0. Lo stesso principio vale per il flag che abilita il Secure Boot, per la chiave di Flash Encryption e per il contatore anti-rollback ([§ 5.6](#)). L'irreversibilità è la loro forza ed è anche il motivo per cui la procedura di produzione va provata con attenzione su schede di test prima di toccare un lotto vero. La procedura di programmazione è in [§ 4.5](#).

4. Secure Boot

4.1 In una frase

Secure Boot

Al reset, il codice immutabile in ROM verifica la firma del bootloader; il bootloader verifica la firma dell'applicazione; nessuno stadio cede il controllo a codice la cui firma non è stata validata con la chiave pubblica il cui digest è in eFuse. Il risultato è una **catena di fiducia** che parte dal silicio. Funzionamento dettagliato in § 4.4, procedura in § 4.5.

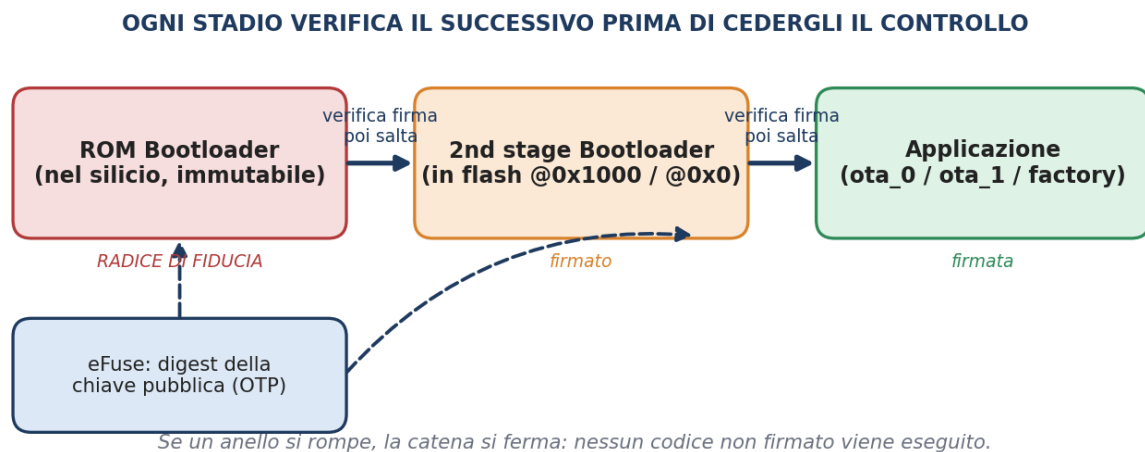


Figura 5 — Catena di fiducia su ESP32. La radice è il bootloader in ROM, che non può essere modificato perché fa parte del chip. Ogni anello verifica il successivo prima di saltarvi. La chiave pubblica di riferimento è ancorata in eFuse.

4.2 Perché implementarlo

Il Secure Boot è la contromisura alle minacce «fisiche» della figura 2. In concreto impedisce che:

- un firmware modificato (anche di un solo byte, per esempio per alterare un coefficiente di calibrazione o disattivare un controllo) venga eseguito;
- un firmware di terzi venga caricato su hardware Carlo Gavazzi per riutilizzarlo o per farne un vettore d'attacco verso la rete del cliente;
- un firmware legittimo ma **vecchio** venga ripristinato per sfruttare una vulnerabilità già corretta (con anti-rollback, § 5.6);
- le interfacce di debug (JTAG, UART download mode) restino aperte sul campo: in modalità «release» il Secure Boot le disabilita via eFuse. → § 4.4

C'è anche una ragione di processo: senza Secure Boot, ogni altra misura di sicurezza software (TLS, autenticazione Modbus/TCP, cifratura dei dati) può essere rimossa da chi riscrive la flash. Il Secure Boot è il fondamento su cui le altre poggiano.

4.3 Quando implementarlo

Situazione	Secure Boot?	Note
Prodotto connesso in rete (Wi-Fi, Ethernet, BLE) venduto nell'UE	Sì, obbligo di fatto	CRA Allegato I richiede protezione dell'integrità del software; RED-DA per i prodotti radio (§ 7)

Situazione	Secure Boot?	Note
Prodotto con misura fiscale o metrologica	Sì	L'integrità del software di misura è già richiesta dalla MID/WELMEC; il Secure Boot è il modo moderno di dimostrarla
Prodotto non connesso, in ambiente controllato	Consigliato	Protegge IP e calibrazione; costo quasi nullo se il flusso di produzione lo prevede fin dall'inizio
Scheda di sviluppo / prototipo interno	No, ma progettare per averlo	Usare modalità development con chiave dev (§ 4.4); non bruciare eFuse irreversibili
Prodotto già sul campo senza Secure Boot	Non retrofittabile	Il digest chiave e SECURE_BOOT_EN vanno scritti in fabbrica; non si abilitano via OTA in sicurezza

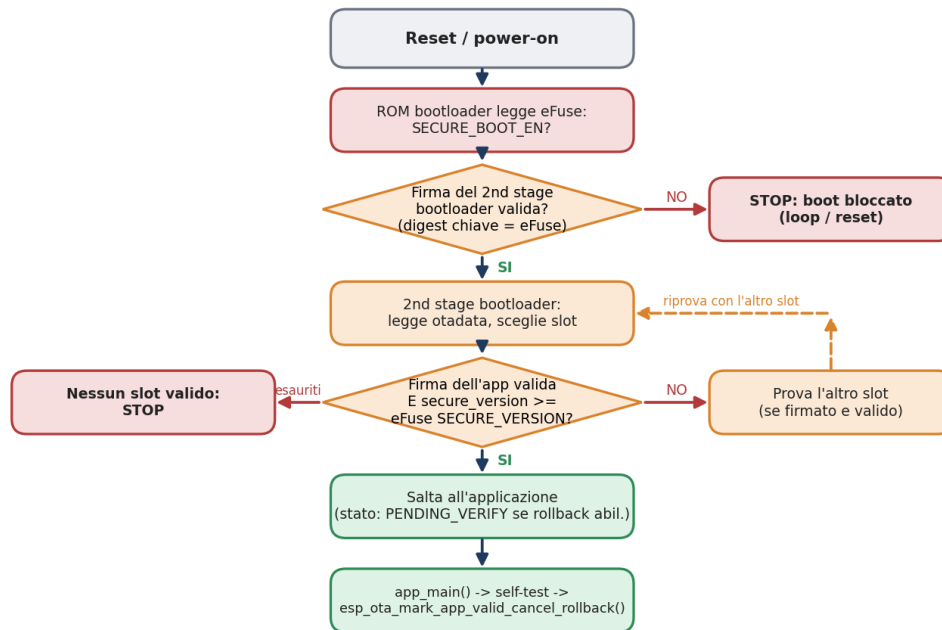
Il momento giusto è l'inizio

Il Secure Boot si decide alla definizione del prodotto, non alla fine. Condiziona: il layout delle partizioni (spazio per i signature block, offset della partition table), il flusso di produzione (bruciatura eFuse, chiavi), il processo di build (chi firma, [§ 6](#)), la scelta del chip (ESP32 rev. ECO3 o successive per Secure Boot V2). Aggiungerlo dopo costa ordini di grandezza in più.

4.4 Come funziona su ESP32: Secure Boot V2

Espressif ha avuto due schemi. **Secure Boot V1** (solo ESP32 originale, basato su AES e un segreto simmetrico nel chip) è superato e non va usato per nuovi progetti. **Secure Boot V2** è lo schema attuale per ESP32 (revisione silicio ECO3 o successive), ESP32-S2, S3, C3, C6, H2, P4 e successivi: usa firme **RSA-3072 con padding PSS** (o ECDSA P-256 su alcuni chip recenti, per esempio ESP32-C2) e verifica l'immagine con la chiave pubblica il cui digest SHA-256 è in eFuse.

Il flusso di verifica è il seguente:

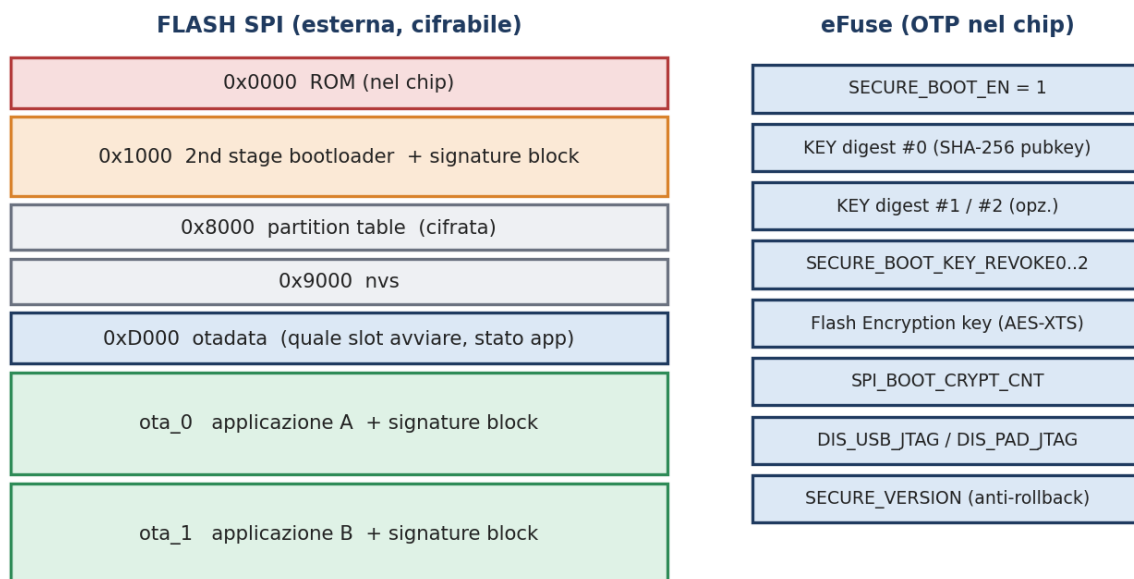


Se l'app non conferma entro il primo boot, al reset il bootloader torna allo slot precedente (rollback).

Figura 6 — Flusso di avvio con Secure Boot V2, rollback e anti-rollback abilitati. I rombi arancioni sono i punti di decisione; i riquadri rossi sono gli stati terminali di errore. Il ramo «prova l'altro slot» esiste solo con lo schema OTA a doppia partizione (§ 5.4, § 5.5); il controllo di `secure_version` è l'anti-rollback (§ 5.6).

Punti tecnici da fissare:

- **Il ROM bootloader** è la radice di fiducia. Legge `SECURE_BOOT_EN` in eFuse; se è a 1, verifica la firma del bootloader di secondo stadio prima di eseguirlo.
- **Il signature block** è una struttura di 1216 byte appesa in coda a ogni immagine (bootloader e app), allineata a 4 KB. Contiene la chiave pubblica completa, la firma RSA-PSS del digest dell'immagine e un CRC. Fino a **3 blocchi** possono essere appesi alla stessa immagine, firmati con chiavi diverse: è il meccanismo che rende possibile la rotazione delle chiavi (esempio qui sotto e in § 6.2).
- **Il digest in eFuse**: su ESP32 (ECO3) c'è un solo slot (BLOCK2); su S2/S3/C3 e successivi ce ne sono tre (`SECURE_BOOT_DIGEST0..2`), ognuno con un bit di revoca (`SECURE_BOOT_KEY_REVOKE0..2`). L'immagine è accettata se almeno una delle sue firme corrisponde a un digest presente e non revocato.
- **Il bootloader di secondo stadio** (quello di ESP-IDF, in flash) verifica l'applicazione con lo stesso schema, poi salta a `app_main()`.



Il 'signature block' (1216 B) è appeso in coda a ogni immagine, allineato a 4 KB; fino a 3 blocchi per immagine.

Figura 7 — A sinistra il layout tipico della flash con schema OTA a due slot; ogni immagine eseguibile ha il suo signature block in coda. A destra gli eFuse coinvolti: sono OTP, quindi vanno programmati una volta sola e nell'ordine giusto (procedura in § 4.5). Le partizioni ota_0/ota_1/otadata sono spiegate in § 5.4.

Perché «fino a 3 blocchi»: un esempio

La stessa immagine può portare non una ma fino a tre firme, ognuna prodotta con una chiave privata diversa. I blocchi stanno uno dietro l'altro nel settore da 4 KB in coda al firmware ($3 \times 1216 = 3648$ byte) e firmano tutti lo stesso hash dell'immagine: cambia solo la coppia chiave pubblica + firma dentro ciascun blocco. Il bootloader li scorre in ordine e accetta l'immagine se **almeno uno** ha una chiave il cui digest è in eFuse e non è revocato.

In fabbrica si programmano tre digest: K0 (la chiave in uso), K1 e K2 (riserva, custodite ma mai usate). Tutti i firmware hanno un solo blocco, firmato con K0. Un giorno K0 viene compromessa: gli eFuse dei dispositivi sul campo non si possono cambiare, ma si può fare questo:

- Si rilascia il firmware 3.0 con **due blocchi**: uno firmato con K0, uno con K1.
- I dispositivi lo accettano grazie al blocco K0, che per loro è ancora valida.
- Il firmware 3.0, una volta partito e confermato, brucia l'eFuse SECURE_BOOT_KEY_REVOKE0: da quel momento il dispositivo ignora qualsiasi blocco firmato con K0.
- Dalla 3.1 in avanti si firma solo con K1, con un blocco solo. Chi possiede K0 non può più far partire nulla.

Senza il secondo blocco il primo passo sarebbe impossibile: un firmware firmato solo con K1 verrebbe rifiutato dai dispositivi che conoscono solo K0, e uno firmato solo con K0 non potrebbe «traghetare» verso la chiave nuova. § 6.2 riprende le conseguenze pratiche: le chiavi di riserva devono esistere prima della produzione, e sull'ESP32 originale (un solo digest, nessuna revoca) la rotazione non è possibile.

Modalità development e modalità release

ESP-IDF distingue due modalità, e la differenza è tutta negli eFuse aggiuntivi che vengono bruciati:

	Development (CONFIG_SECURE_BOOT_INSECURE)	Release (default)
Verifica firma	Sì	Sì
JTAG	Resta abilitato	Disabilitato via eFuse
UART download mode	Abilitato (si può riflashare)	Disabilitato o «secure download mode» (solo comandi limitati)
Lettura eFuse chiave	Possibile	Protetta in lettura/scrittura
Aggiunta di nuovi digest chiave	Possibile	Bloccata (SECURE_BOOT_AGGRESSIVE_REVOKE opz.)
Uso	Schede di sviluppo, test del flusso	Produzione

Irreversibile

Una volta bruciato SECURE_BOOT_EN in modalità release, la scheda accetta **per sempre** solo firmware firmato con una chiave il cui digest è in eFuse. Se si perde la chiave privata, quel dispositivo non è più aggiornabile: né via OTA né via cavo. Provare l'intera procedura su almeno una decina di schede sacrificabili, e leggere sempre l'output di `espefuse.py summary` prima e dopo. Custodia e backup della chiave: [§ 6.1](#).

4.5 Come abilitarlo: procedura passo-passo

La procedura seguente si riferisce a ESP-IDF v5.x. I nomi dei comandi e delle opzioni possono variare tra versioni: verificare sempre sulla documentazione della versione in uso prima di applicarla su hardware reale.

Passo 1 — Generare la chiave di firma (una volta, in ambiente protetto)

```
# RSA-3072 per Secure Boot V2 (ESP32 EC03, S2, S3, C3, C6, ...)
esptool.py generate_signing_key --version 2 --scheme rsa3072 secure_boot_signing_key.pem

# Estrarre la chiave pubblica (per firma remota / verifica esterna)
esptool.py extract_public_key --version 2 --keyfile secure_boot_signing_key.pem pub_key.pem

# Calcolare il digest che finirà in eFuse
esptool.py digest_sbv2_public_key --keyfile secure_boot_signing_key.pem --output digest.bin
```

Per la produzione la chiave va generata **dentro un HSM o un servizio KMS**, non su un PC di sviluppo: vedi capitolo 6. In quel caso `generate_signing_key` non si usa e si lavora solo con la chiave pubblica esportata. → § 6.1

Passo 2 — Configurare il progetto (menuconfig → Security features)

```
CONFIG_SECURE_BOOT=y
CONFIG_SECURE_BOOT_V2_ENABLED=y
CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES=y          # firma automatica in build (solo con chiave locale)
CONFIG_SECURE_BOOT_SIGNING_KEY="secure_boot_signing_key.pem"
# CONFIG_SECURE_BOOT_INSECURE=y                      # SOLO su schede di sviluppo
CONFIG_SECURE_FLASH_ENC_ENABLED=y                   # Flash Encryption (consigliato insieme)
CONFIG_SECURE_FLASH_ENCRYPTION_MODE_RELEASE=y       # in produzione
CONFIG_PARTITION_TABLE_OFFSET=0x10000                # se il bootloader firmato supera 0x8000
```

Il bootloader con Secure Boot e Flash Encryption è più grande di quello standard: spesso non entra nello spazio tra 0x1000 e 0x8000 dove per default inizia la partition table. Spostare l'offset della partition table (per esempio a 0x10000) è la correzione tipica al primo errore di build «bootloader binary size exceeds...».

Passo 3 — Costruire e flashare bootloader firmato e applicazione firmata

```
idf.py bootloader          # produce build/bootloader/bootloader.bin già firmato
idf.py -p PORT bootloader-flash # con Secure Boot abilitato, 'idf.py flash' NON flasha il bootloader
idf.py build
idf.py -p PORT flash       # partition table + app (firmata) [+ otadata]
```

Passo 4 — Programmare gli eFuse (produzione)

```
# ESP32 (EC03): un solo digest, in BLOCK2
esptool.py -p PORT burn_key secure_boot_v2 digest.bin

# ESP32-S3 / C3 / C6: fino a tre digest con KEY_PURPOSE dedicato
esptool.py -p PORT burn_key BLOCK_KEY0 digest.bin SECURE_BOOT_DIGEST0

# Abilitare il Secure Boot (irreversibile)
esptool.py -p PORT burn_efuse SECURE_BOOT_EN

# Verifica finale
esptool.py -p PORT summary
```

In alternativa al comando manuale, al primo boot il bootloader in modalità «enable on first boot» può bruciare da solo il digest della chiave contenuta nel suo signature block e il flag `SECURE_BOOT_EN`. È comodo in sviluppo, ma in produzione è preferibile una procedura esplicita e loggata da un tool di fabbrica, così che ogni eFuse scritto sia tracciato.

Passo 5 — Verificare il comportamento negativo

Un test di sicurezza non è completo se si verifica solo che il firmware firmato parte. Vanno provati anche i casi di rifiuto: flashare un'immagine non firmata e osservare che il boot si ferma; alterare un byte dell'immagine firmata; firmare con una chiave diversa. Il log seriale del ROM bootloader riporta il motivo del blocco. Questi tre test vanno automatizzati nella validazione di ogni release del bootloader. Sono ripresi nella checklist, [§ 8.4](#).

4.6 Flash Encryption: il complemento naturale

La Flash Encryption cifra in modo trasparente il contenuto della flash SPI con una chiave AES (AES-XTS a 256 bit sui chip recenti) memorizzata in eFuse e non leggibile dal software. La CPU legge la flash attraverso una cache che decifra al volo; il codice applicativo non se ne accorge. Vengono cifrati bootloader, partition table, tutte le partizioni di tipo *app*, otadata e le partizioni dati marcate con il flag encrypted. La partizione NVS invece **non** può usare la Flash Encryption e ha un proprio meccanismo (NVS Encryption) con chiavi in una partizione dedicata.

Cosa aggiunge rispetto al solo Secure Boot: impedisce la lettura del firmware (protezione della proprietà intellettuale e delle eventuali credenziali salvate) e rende inutile la copia della flash da un dispositivo a un altro, perché la chiave è unica per chip se generata dal chip stesso al primo boot. Cosa **non** aggiunge: nessuna garanzia di autenticità. I due meccanismi vanno insieme; nel progetto di produzione la scelta ragionevole è abilitarli entrambi in modalità release ([§ 4.4](#)). Per la cifratura del pacchetto OTA in transito, che è un'altra cosa ancora, vedi [§ 5.9](#).

5. Secure Update

5.1 In una frase

Secure Update

Un aggiornamento firmware (OTA o via cavo) viene installato solo se arriva da un canale autenticato, è firmato con la chiave del produttore, ha una versione di sicurezza non inferiore a quella già installata, e viene attivato in modo atomico con la possibilità di tornare indietro se la nuova versione non funziona. Flusso completo in [§ 5.4](#), rollback in [§ 5.5](#), anti-rollback in [§ 5.6](#).

Il Secure Boot protegge il dispositivo *nel tempo che sta fermo*; il Secure Update lo protegge *mentre cambia*. I due sono complementari: il Secure Boot garantisce che, qualunque cosa sia stata scritta in flash, solo il firmware firmato parte; il Secure Update garantisce che quello che si scrive in flash sia stato scelto, trasportato e attivato correttamente. Un sistema con Secure Boot ma senza un processo di aggiornamento affidabile diventa un sistema non aggiornabile — ed è esattamente il tipo di prodotto che la normativa europea vuole eliminare dal mercato ([§ 7](#)).

5.2 Perché implementarlo

- **Autenticità del pacchetto:** senza firma, chiunque riesca a farsi passare per il server OTA (DNS spoofing, rete del cliente compromessa, server violato) può installare quello che vuole. TLS protegge il canale; la firma protegge il *contenuto* anche se il canale o il server sono compromessi. → [§ 5.8](#)
- **Anti-rollback:** una vulnerabilità corretta nella versione N è ancora sfruttabile se qualcuno può reinstallare la versione N-1. Il contatore monotono in eFuse impedisce il ritorno a versioni con problemi di sicurezza noti. → [§ 5.6](#)
- **Disponibilità:** un aggiornamento interrotto (rete, alimentazione) non deve lasciare un dispositivo inutilizzabile in un quadro elettrico a 200 km di distanza. Il doppio slot con conferma e rollback automatico è la risposta standard. → [§ 5.5](#)
- **Obbligo normativo:** il CRA richiede che le vulnerabilità possano essere corrette con aggiornamenti di sicurezza distribuiti in modo sicuro, e che gli aggiornamenti di sicurezza siano per default automatici e separati da quelli funzionali dove tecnicamente possibile.

5.3 Quando implementarlo

Sempre, se il prodotto ha un canale di comunicazione. Anche un dispositivo che oggi si aggiorna solo via USB in laboratorio beneficia della verifica di firma nell'immagine di update (senza Secure Boot completo, [§ 5.7](#)): costa una partizione in più e poche righe di configurazione. Le domande da porsi non sono «se» ma «quali di queste funzioni»:

Funzione	Serve quando...	Costo
Verifica firma dell'update	Sempre	Un signature block per immagine; tempo di verifica di pochi ms
TLS con verifica certificato server	Sempre, per OTA in rete	Certificato CA nel firmware; gestione scadenza/rotazione
Doppio slot + rollback automatico	Dispositivi non fisicamente raggiungibili	Flash raddoppiata per la parte app (2 × dimensione app + otadata)
Anti-rollback (secure_version)	Prodotto con vita > 3 anni o con esposizione in rete	Un bit eFuse per ogni incremento; disciplina di versioning

Funzione	Serve quando...	Costo
Cifratura del pacchetto OTA	Firmware con IP sensibile o chiavi embedded	Pre-cifratura in fabbrica + decifratura sul dispositivo (esp_encrypted_img)
Aggiornamento differenziale	Banda limitata (cellulare, LoRa)	Complessità significativa; valutare solo se necessario

5.4 Come funziona su ESP32: il flusso OTA

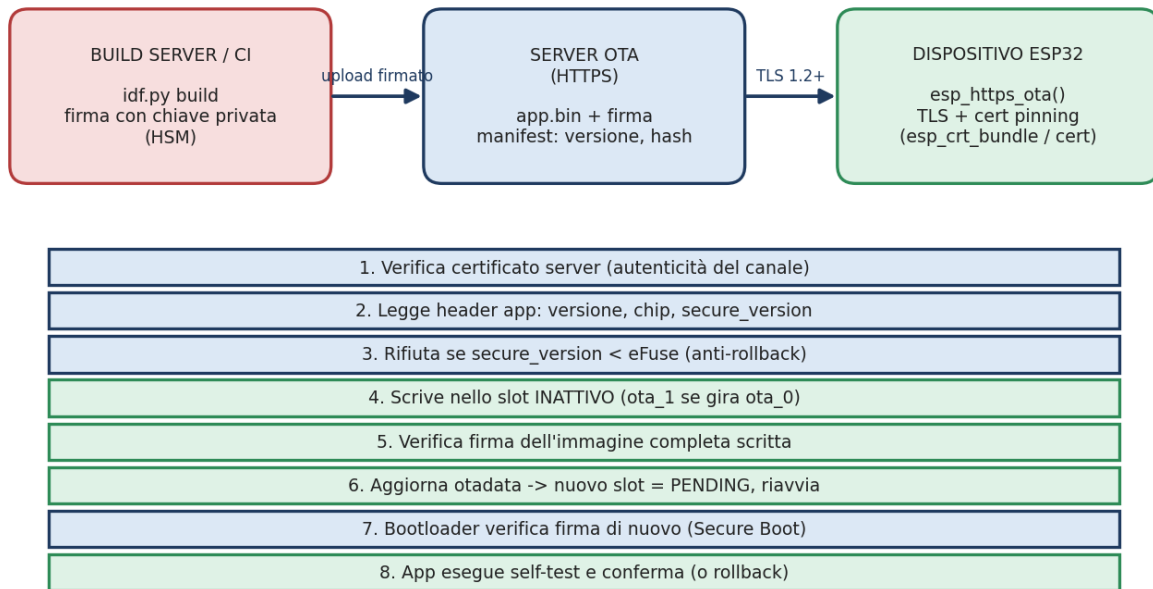


Figura 8 — Flusso di aggiornamento sicuro dal build server al dispositivo. In verde i passi che scrivono o cambiano lo stato della flash; ognuno è preceduto da una verifica. La firma viene controllata due volte: dall'app durante l'OTA e dal bootloader al riavvio.

La sequenza completa, con riferimento alle API ESP-IDF:

- **Connessione:** `esp_https_ota()` apre una sessione HTTPS. Il certificato del server è verificato contro una CA compilata nel firmware (`esp_cert_bundle_attach` per CA pubbliche, oppure un certificato specifico per un server aziendale — preferibile, perché limita l'insieme dei server accettati).
- **Header:** i primi byte scaricati contengono l'`esp_app_desc_t`: versione, nome progetto, data di build, `secure_version`. Il dispositivo può decidere qui di non proseguire (stessa versione, chip diverso, versione di sicurezza inferiore) risparmiando banda.
- **Scrittura:** l'immagine è scritta nello slot OTA *inattivo* (`esp_ota_get_next_update_partition`). L'applicazione in esecuzione non viene toccata.
- **Verifica:** al termine, `esp_ota_end()` verifica l'integrità e, con Secure Boot V2 o con la sola verifica di firma abilitata, la firma del signature block. Se fallisce, lo slot resta invalido e non viene mai avviato. Struttura del signature block in [§ 4.4](#).
- **Attivazione:** `esp_ota_set_boot_partition()` aggiorna `otadata` con il nuovo slot nello stato *new*; poi `esp_restart()`.
- **Conferma:** al riavvio il bootloader verifica di nuovo la firma (Secure Boot) e avvia la nuova app in stato *pending verify*. L'app esegue i propri controlli e chiama `esp_ota_mark_app_valid_cancel_rollback()`. Se non lo fa prima del prossimo reset, il bootloader torna allo slot precedente. → [§ 5.5](#)

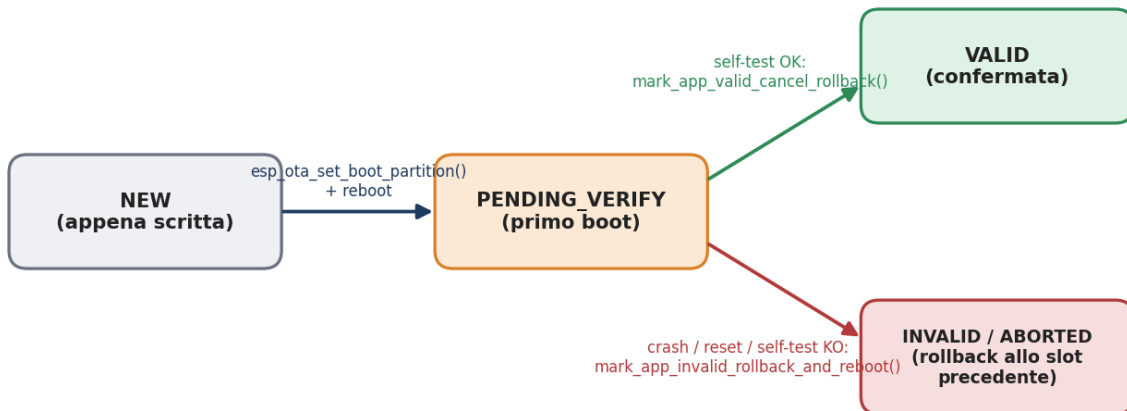
Layout delle partizioni

```
# partitions.csv – schema OTA a due slot (esempio, flash 4 MB)
# Name,      Type, SubType, Offset,  Size,      Flags
nvs,         data, nvs,      ,       0x6000,
otadata,     data, ota,      ,       0x2000,
phy_init,    data, phy,      ,       0x1000,
ota_0,       app,  ota_0,    ,       0x1E0000,
```

```
ota_1,      app, ota_1,      ,      0x1E0000,  
nvs_key,    data, nvs_keys, ,      0x1000, encrypted
```

La partizione otadata (8 KB, due settori per ridondanza) contiene il numero di sequenza e lo stato di ogni slot: è ciò che il bootloader legge per decidere quale app avviare. È cifrata dalla Flash Encryption ma non firmata; la sua manomissione può al massimo far avviare l'altro slot (comunque firmato), non codice arbitrario.

5.5 Rollback: stati dell'applicazione



Il bootloader gestisce il caso peggiore: se l'app in PENDING_VERIFY si resetta senza confermare, la marca ABORTED e torna indietro.

Figura 9 — Macchina a stati di uno slot OTA con `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`. La transizione critica è quella da PENDING_VERIFY: se l'app non conferma esplicitamente, il bootloader considera l'aggiornamento fallito.

Cosa deve fare l'applicazione al primo avvio dopo un update, prima di confermare:

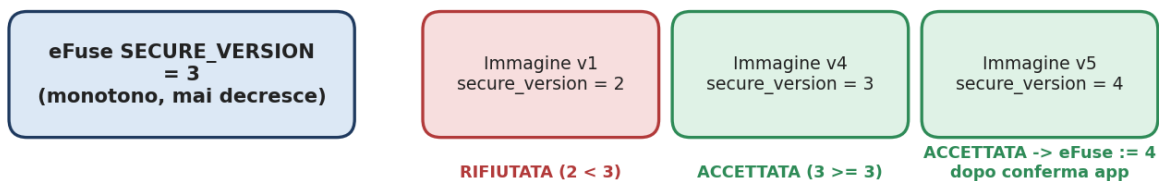
- verificare che la connettività essenziale funzioni (per esempio: ottenuto IP, raggiunto il broker/server, ricevuto un ACK);
- verificare che i dati persistenti siano stati migrati correttamente (versione NVS, calibrazione letta e plausibile);
- eseguire i self-test hardware che il prodotto già prevede (ADC di misura, RTC, memoria esterna);
- solo allora chiamare `esp_ota_mark_app_valid_cancel_rollback()`. In caso di fallimento, chiamare `esp_ota_mark_app_invalid_rollback_and_reboot()`, che torna subito allo slot precedente.

```
// Nel task principale, dopo l'inizializzazione:
const esp_partition_t *running = esp_ota_get_running_partition();
esp_ota_img_states_t state;
if (esp_ota_get_state_partition(running, &state) == ESP_OK &&
    state == ESP_OTA_IMG_PENDING_VERIFY) {
    if (self_test_ok()) {
        esp_ota_mark_app_valid_cancel_rollback(); // conferma: questa versione resta
    } else {
        esp_ota_mark_app_invalid_rollback_and_reboot(); // torna allo slot precedente
    }
}
```

Il caso che non si vede

Il rollback protegge anche dal caso in cui la nuova app entra in crash-loop prima di arrivare al self-test: il watchdog o il panic handler resettano il chip, il bootloader vede uno slot ancora in PENDING_VERIFY dopo un reset e lo marca ABORTED. Per questo la conferma deve avvenire **dopo** aver esercitato il codice che potrebbe fallire, non nelle prime righe di `app_main()`.

5.6 Anti-rollback: la versione di sicurezza



Incrementa secure_version SOLO quando l'aggiornamento chiude una vulnerabilità di sicurezza, non a ogni release: ogni incremento brucia un bit eFuse (risorsa finita: 32 bit su ESP32, 16 su S2/S3/C3).

Nota: il valore massimo dipende dal chip e va verificato sulla documentazione ESP-IDF del target.

Figura 10 — Il contatore secure_version in eFuse accetta solo immagini con versione di sicurezza uguale o superiore. Il bootloader lo aggiorna (bruciando i bit) solo dopo che la nuova app è stata confermata valida.

Il numero di versione «normale» (1.4.2, 1.4.3...) non basta contro il downgrade: è solo una stringa nell'header e nulla impedisce di installare la 1.4.2 dopo la 1.4.3. L'anti-rollback introduce un secondo numero, secure_version, impostato con CONFIG_BOOTLOADER_APP_SECURE_VERSION e incluso nell'app descriptor firmato. Il bootloader rifiuta le immagini con secure_version minore del valore in eFuse, e dopo la conferma di un'app con valore superiore brucia i bit necessari per allineare l'eFuse. Sugli eFuse come memoria OTP vedi [§ 3.4](#).

- Attivazione: CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y (richiede rollback abilitato e, in pratica, Secure Boot: senza firma il campo si può falsificare).
- Il contatore è una risorsa finita: 32 valori su ESP32, 16 su S2/S3/C3 (verificare per il chip in uso). Va incrementato **solo** quando la release chiude una vulnerabilità che non deve più essere sfruttabile, non a ogni build.
- Conseguenza organizzativa: la decisione di incrementare secure_version è una decisione di prodotto da documentare nel changelog di sicurezza, non un automatismo della CI.
- Con l'anti-rollback il campo ota_0/ota_1 «vecchio» smette di essere un backup utilizzabile appena l'eFuse viene incrementato: il rollback automatico funziona solo *prima* della conferma.

5.7 Verifica di firma senza Secure Boot completo

ESP-IDF offre una via intermedia utile in due casi: prodotti già sul campo senza Secure Boot, e fasi di sviluppo in cui non si vogliono ancora bruciare eFuse. Con `CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT=y` e `CONFIG_SECURE_SIGNED_ON_UPDATE_NO_SECURE_BOOT=y`, la libreria OTA verifica la firma delle immagini ricevute usando una chiave pubblica compilata nel bootloader, senza che il ROM bootloader partecipi. Protegge dagli aggiornamenti OTA non autorizzati, ma non da chi riscrive la flash via cavo (quello richiede il Secure Boot vero, § 4.4). È un buon primo passo, non un sostituto.

5.8 Il canale: TLS e pinning

La firma rende superfluo fidarsi del server per l'*integrità* del firmware, ma il canale TLS resta necessario per altre ragioni: confidenzialità del firmware in transito (se non è pre-cifrato), autenticazione del dispositivo verso il server (certificato client o token, per evitare che chiunque scarichi le immagini), protezione da manipolazioni del manifest. Raccomandazioni pratiche:

- Usare un certificato CA **specifico** del server aziendale invece del bundle di CA pubbliche, dove possibile: riduce la superficie a un solo emittente.
- Pianificare fin dall'inizio la **rotazione dei certificati**: un dispositivo con vita di 10 anni vedrà scadere ogni certificato in flash. Il meccanismo tipico è consegnare la nuova CA con un aggiornamento firmware prima della scadenza della vecchia.
- Non disabilitare mai la verifica del certificato «temporaneamente» in produzione (`skip_cert_common_name_check` e simili): sono le opzioni che poi restano nel codice.
- Per l'autenticazione del dispositivo preferire certificati client per dispositivo (provisioning in fabbrica) alle credenziali condivise.

5.9 Cifratura del pacchetto OTA

Se il firmware contiene proprietà intellettuale da proteggere, la Flash Encryption protegge il dato a *riposo* nel chip ma il pacchetto OTA viaggia in chiaro dentro TLS e resta in chiaro sul server e nei log. Il componente `esp_encrypted_img` (repository `esp-idf-extra` / IDF component registry) permette di pre-cifrare l'immagine con una chiave RSA/AES e decifrarla sul dispositivo durante la scrittura in flash. Si usa quando il modello di minaccia lo richiede; non è necessario per l'autenticità, che è già garantita dalla firma.

6. Gestione delle chiavi di firma

Tutto quanto descritto finora vale esattamente quanto la protezione della chiave privata. Se la chiave privata viene copiata, l'attaccante può firmare firmware che tutti i dispositivi sul campo accetteranno, e non c'è patch software che possa rimediare, perché il dispositivo non ha modo di distinguere la firma legittima da quella dell'attaccante. La gestione delle chiavi è quindi un problema organizzativo prima che tecnico. Il meccanismo di firma è spiegato in [§ 3.2](#); come la chiave pubblica arriva nel dispositivo (signature block, digest in eFuse) in [§ 4.4](#) e [§ 4.5](#).

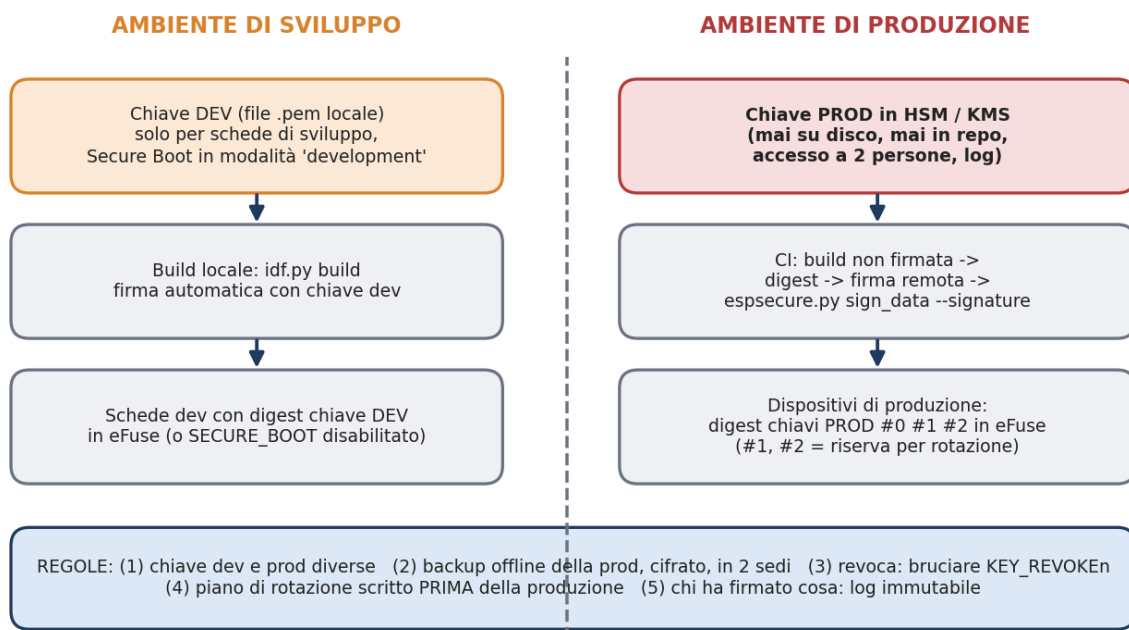


Figura 11 — Separazione tra ambiente di sviluppo (chiave dev, bassa criticità) e produzione (chiave prod in HSM, firma remota). I tre digest in eFuse dei dispositivi di produzione sono la riserva per la rotazione.

6.1 Principi

- **Due chiavi, due mondi:** una chiave di sviluppo che può stare in un file (anche in un repo privato) e una chiave di produzione che non deve mai esistere in chiaro su un disco. Le schede di sviluppo hanno il digest della chiave dev; quelle di produzione quello della chiave prod. Un firmware firmato dev non parte mai su un dispositivo di produzione, e viceversa.
- **Chiave prod in HSM o KMS:** la chiave viene generata dentro un modulo hardware o un servizio cloud (Azure Key Vault, AWS KMS, YubiHSM, ecc.) e non è esportabile. Si esporta solo la chiave pubblica.
- **Firma remota:** la CI produce l'immagine non firmata, ne calcola il digest e lo invia al servizio di firma; riceve la firma e la appende. Espressif supporta il flusso con `espsecure.py sign_data --version 2 --pub-key pub.pem --signature sig.bin`.
- **Controllo degli accessi e log:** chi può chiedere una firma, con quale approvazione, e un registro immutabile di quali digest sono stati firmati e quando. In caso di incidente è l'unico modo di sapere cosa è stato firmato.
- **Backup:** se l'HSM lo consente, backup cifrato della chiave in almeno due sedi fisiche separate, con procedura di ripristino provata. La perdita della chiave prod rende non aggiornabile l'intero parco installato.

6.2 Rotazione e revoca

Su ESP32-S2/S3/C3 e successivi si programmano fin dalla fabbrica **tre digest**: la chiave prod #0 in uso e due chiavi di riserva (#1, #2) generate e custodite ma mai usate. Se la chiave #0 viene compromessa, si pubblica un aggiornamento firmato con **entrambe** #0 e #1 (due signature block): i dispositivi lo accettano grazie a #0, e la nuova app brucia il bit SECURE_BOOT_KEY_REVOKE0. Da quel momento solo #1 è valida. Un esempio passo-passo dello stesso scenario è in [§ 4.4](#). Le regole:

- Le chiavi di riserva devono esistere **prima** della produzione: non si possono aggiungere digest a un dispositivo in modalità release.
- Il firmware di transizione deve portare più firme, quindi il layout deve prevedere spazio per fino a 3 signature block (3 × 1216 byte, allineati a 4 KB).
- Su ESP32 originale (ECO3) c'è un solo digest e nessuna revoca: la compromissione della chiave non è recuperabile. Per un prodotto nuovo è un argomento a favore di S3/C3/C6.
- Scrivere la procedura di rotazione e provarla in laboratorio come parte della validazione del prodotto, non dopo l'incidente.

6.3 Cosa NON fare

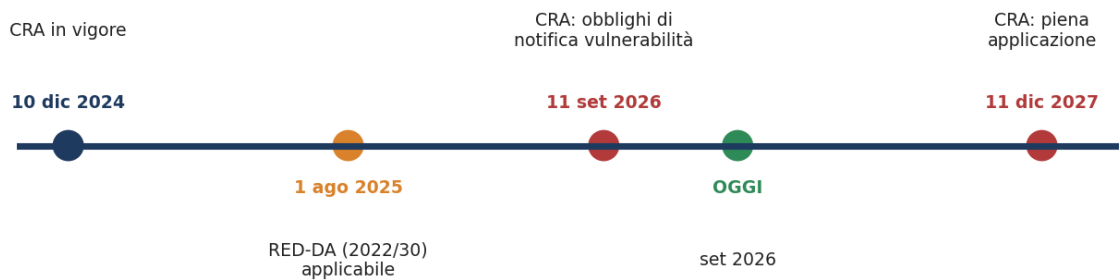
Anti-pattern visti sul campo

- Chiave di firma nel repository del firmware, «tanto è privato».
- Stessa chiave per Secure Boot e per il certificato TLS o per la firma di altri prodotti.
- Chiave dev bruciata nei dispositivi di produzione perché «la prod non era pronta».
- Firma manuale da parte di uno sviluppatore sul proprio laptop per una release urgente.
- Nessun inventario di quali lotti hanno quale digest in eFuse.

7. Riferimenti normativi: CRA e RED-DA

Avvertenza sulla verifica

Questo capitolo riporta il contenuto dei regolamenti come lo conosco; al momento della stesura non avevo accesso alla rete per riscontrare articoli, allegati e date sul testo ufficiale. Per qualsiasi uso in documentazione di conformità, verificare sul testo consolidato in EUR-Lex (Reg. (UE) 2024/2847 e Reg. delegato (UE) 2022/30) e sulle norme armonizzate EN 18031-1/2/3.



Date da verificare sul testo ufficiale EUR-Lex (riportate a memoria, senza accesso web al momento della stesura).

Figura 12 — Collocazione temporale. RED-DA è già applicabile; il CRA ha una fase di obblighi di notifica dal settembre 2026 e la piena applicazione dei requisiti essenziali dal dicembre 2027. Date da verificare sul testo ufficiale.

7.1 Cyber Resilience Act — Reg. (UE) 2024/2847

Il CRA introduce requisiti essenziali di cybersicurezza per tutti i «prodotti con elementi digitali» immessi sul mercato UE. Un energy meter connesso vi rientra. I requisiti si trovano nell'**Allegato I**, diviso in due parti: la Parte I riguarda le proprietà di sicurezza del prodotto, la Parte II la gestione delle vulnerabilità. Quelli che Secure Boot e Secure Update soddisfano direttamente sono, in sintesi e con il mio riassunto (non il testo legale):

Requisito (Allegato I, riassunto)	Meccanismo	Nota
Protezione dell'integrità di software, dati, comandi e configurazione contro manipolazioni non autorizzate (Parte I)	Secure Boot; firma OTA	Il firmware è parte del «software» del prodotto (§ 4)
Protezione della confidenzialità dei dati memorizzati (Parte I)	Flash Encryption; NVS Encryption	Rilevante se in flash ci sono chiavi, credenziali, dati personali
Il prodotto deve essere immesso senza vulnerabilità note sfruttabili e con configurazione sicura per default (Parte I)	Modalità release: JTAG e download mode disabilitati	Il debug aperto in produzione è una vulnerabilità nota (§ 4.4)
Le vulnerabilità devono poter essere corrette con aggiornamenti di sicurezza, distribuiti in modo sicuro, per default automatici e con notifica all'utente (Parte I e II)	Secure Update completo: firma, TLS, rollback, anti-rollback	Il periodo di supporto atteso è di almeno 5 anni salvo eccezioni (§ 5)
Meccanismi per registrare e monitorare attività interne rilevanti (Parte I)	Log di boot, esito verifiche, tentativi OTA falliti	Da esporre verso la piattaforma di gestione
Gestione delle vulnerabilità: SBOM, politica di divulgazione coordinata, notifica a ENISA delle vulnerabilità attivamente sfruttate (Parte II)	Processo organizzativo	Non è firmware, ma il Secure Update è il suo strumento operativo

La logica del CRA è che senza una catena di fiducia il produttore non può *dimostrare* di controllare il software in esecuzione sui propri dispositivi, e senza un aggiornamento sicuro non può adempiere all'obbligo di correggere le vulnerabilità per tutto il periodo di supporto. Secure Boot e Secure Update non sono nominati esplicitamente nel regolamento (che è neutrale rispetto alla tecnologia), ma sono lo stato dell'arte con cui i requisiti vengono soddisfatti.

7.2 RED Delegated Act — Reg. delegato (UE) 2022/30

Il RED-DA attiva, per le apparecchiature radio connesse a Internet (quindi ogni dispositivo con Wi-Fi o BLE), i requisiti essenziali dell'art. 3(3) lettere d), e), f) della Direttiva 2014/53/UE: non danneggiare la rete, proteggere i dati personali e la privacy, proteggere dalla frode. Le norme armonizzate di riferimento sono la serie **EN 18031** (-1 per la rete, -2 per la privacy, -3 per la frode). Queste norme contengono requisiti espliciti su meccanismi di aggiornamento sicuro (SUM, Secure Update Mechanism) e sull'integrità del software, valutati con alberi decisionali che il produttore compila nel fascicolo tecnico. Un dispositivo ESP32 con Secure Boot e Secure Update come descritti in questo documento fornisce le evidenze tecniche necessarie per rispondere a quelle domande.

7.3 Come usarli nel progetto

- Mappare fin dalla fase di specifica ogni requisito dell'Allegato I del CRA su un meccanismo tecnico e su un test di verifica; questa tabella diventa parte del fascicolo tecnico.
- Trattare gli eFuse programmati in fabbrica (digest, SECURE_BOOT_EN, flash encryption, revoche) come parametri di configurazione tracciati per lotto.
- Definire il «periodo di supporto» del prodotto e verificare che il piano di rotazione di chiavi e certificati lo copra.
- Per i riferimenti di implementazione sicura (TLS, gestione segreti, logging) usare la OWASP Cheat Sheet Series come riferimento primario, come da linea guida aziendale.

8. Checklist implementativa

Una lista da spuntare, divisa per fase. È volutamente sintetica: ogni voce rimanda a una sezione del documento.

8.1 Progettazione

	Voce	Rif.
☐	Scelto un chip con Secure Boot V2 e almeno 3 slot digest (S3/C3/C6...), o ESP32 rev. ≥ ECO3 accettando i limiti	§ 4.4 , § 6.2
☐	Flash dimensionata per 2 slot app + otadata + spazio per 3 signature block per immagine	§ 5.4 , § 6.2
☐	Offset partition table adeguato al bootloader firmato+cifrato (es. 0x10000)	§ 4.5
☐	Definito il modello di minaccia del prodotto e la mappatura sui requisiti CRA / EN 18031	§ 2 , § 7
☐	Definiti periodo di supporto, piano di rotazione chiavi e certificati	§ 6.2 , § 5.8
☐	Deciso il criterio di incremento di secure_version (chi, quando, dove si documenta)	5.6

8.2 Chiavi e infrastruttura

	Voce	Rif.
☐	Chiave dev generata e distribuita al team; chiave prod generata in HSM/KMS, non esportabile	§ 6.1
☐	Chiavi di riserva #1 e #2 generate e custodite; digest calcolati	§ 6.2
☐	Pipeline CI con firma remota; nessuna chiave privata negli artefatti o nei log	§ 6.1
☐	Registro delle firme e controllo degli accessi al servizio di firma	§ 6.1
☐	Backup cifrato della chiave prod in due sedi, ripristino provato	§ 6.1
☐	Server OTA con certificato dedicato; CA inclusa nel firmware; procedura di rinnovo	5.8

8.3 Firmware

	Voce	Rif.
☐	CONFIG_SECURE_BOOT_V2_ENABLED, Flash Encryption release, ROM download mode disabilitato o secure	§ 4.5 , § 4.6
☐	CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE e ANTI_ROLLBACK attivi; secure_version impostato	§ 5.5 , § 5.6
☐	Self-test post-update implementato; conferma chiamata solo a fine self-test	§ 5.5
☐	OTA via esp_https_ota con verifica certificato; nessuna opzione di skip nel build di release	§ 5.8
☐	Log seriale del bootloader ridotto in release, ma esito delle verifiche esportato verso il backend	§ 7.1
☐	NVS Encryption abilitata se in NVS ci sono credenziali	4.6

8.4 Produzione e validazione

	Voce	Rif.
☐	Procedura eFuse scritta, automatizzata nel tool di fabbrica, con espfuse.py summary salvato per ogni unità	§ 4.5
☐	Procedura provata su schede sacrificabili, inclusi i casi di rifiuto (non firmato, byte alterato, chiave errata)	§ 4.5
☐	Test OTA completo: successo, interruzione a metà, immagine corrotta, downgrade, crash post-update → rollback	§ 5.4-5.6
☐	Test di rotazione chiave (#0 → #1 con revoca) eseguito almeno una volta in laboratorio	§ 6.2
☐	Inventario lotto → digest in eFuse → versione bootloader	§ 6.3
☐	Verificati JTAG e UART download mode effettivamente disabilitati su un'unità di produzione	4.4

9. Errori comuni e come riconoscerli

Sintomo	Causa probabile	Rimedio
Build fallisce: «bootloader binary size exceeds...»	Bootloader con Secure Boot + Flash Encryption più grande dello spazio fino alla partition table	Aumentare CONFIG_PARTITION_TABLE_OFFSET; ridurre log level del bootloader
Dopo idf.py flash il dispositivo non parte più	Con Secure Boot il bootloader non viene flashato da idf.py flash; oppure flashata immagine non firmata	idf.py bootloader-flash; verificare che CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES sia attivo
OTA completato ma al riavvio parte la vecchia app	App non ha chiamato mark_app_valid; o secure_version inferiore all'eFuse	Controllare log bootloader; verificare secure_version dell'immagine
Impossibile aggiornare un lotto di schede di test	eFuse bruciati in release con chiave dev poi persa o ruotata	Non recuperabile: usare sempre schede sacrificabili per provare la procedura
OTA fallisce con errore TLS dopo mesi/anni	Certificato server o CA scaduto	Piano di rotazione; monitorare scadenze; consegnare nuova CA prima della scadenza
Firmware leggibile con esptool nonostante Secure Boot	Secure Boot non è cifratura	Abilitare Flash Encryption in modalità release
Dispositivi diversi hanno lo stesso contenuto flash cifrato	Chiave Flash Encryption pre-generata e condivisa (modalità con chiave host)	Preferire generazione della chiave sul chip al primo boot; se pre-generata, una per dispositivo
Rollback non funziona dopo l'incremento di secure_version	Comportamento atteso: anti-rollback blocca l'immagine vecchia	Confermare la nuova app solo dopo self-test completo; il rollback esiste solo prima della conferma

10. Glossario

Termine	Definizione
Anti-rollback	Meccanismo che impedisce l'installazione di firmware con versione di sicurezza inferiore a quella registrata in eFuse.
App descriptor (esp_app_desc_t)	Struttura all'inizio dell'immagine ESP-IDF con versione, nome progetto, data di build, secure_version. È firmata insieme al resto dell'immagine.
Catena di fiducia (chain of trust)	Sequenza di componenti in cui ciascuno verifica il successivo prima di eseguirlo, a partire da una radice immutabile.
CRA	Cyber Resilience Act, Reg. (UE) 2024/2847: requisiti di cybersicurezza per i prodotti con elementi digitali.
Digest	Risultato di una funzione di hash; in questo documento quasi sempre SHA-256 (32 byte).
eFuse	Memoria OTP nel chip ESP32: bit programmabili una sola volta. Contiene digest chiavi, flag di sicurezza, chiave di cifratura flash.
EN 18031	Serie di norme armonizzate per il RED-DA (rete, privacy, frode); contiene i requisiti sui meccanismi di aggiornamento sicuro.
Firma digitale	Trasformazione dell'hash di un dato con una chiave privata, verificabile con la corrispondente chiave pubblica; garantisce integrità e autenticità.
Flash Encryption	Cifratura trasparente AES della flash SPI dell'ESP32, con chiave in eFuse non leggibile dal software.
HSM	Hardware Security Module: dispositivo che genera e custodisce chiavi private senza permetterne l'esportazione.
Modalità development / release	Configurazioni di Secure Boot e Flash Encryption: la prima lascia aperti debug e riprogrammazione, la seconda li chiude via eFuse.
NVS Encryption	Cifratura della partizione NVS con chiavi XTS-AES in una partizione dedicata; separata dalla Flash Encryption.
OTA	Over-The-Air: aggiornamento del firmware via rete.
otadata	Partizione che memorizza quale slot OTA avviare e il suo stato (new, pending verify, valid, invalid, aborted).
OTP	One-Time Programmable: memoria scrivibile una sola volta.
PENDING_VERIFY	Stato di uno slot OTA al primo avvio dopo l'aggiornamento, in attesa di conferma da parte dell'applicazione.
Radice di fiducia (RoT)	Il primo elemento della catena, che si assume fidato perché non modificabile: il ROM bootloader e il digest chiave in eFuse.
RED-DA	Delegated Act della Radio Equipment Directive, Reg. delegato (UE) 2022/30: attiva i requisiti di cybersicurezza per le apparecchiature radio connesse.
Revoca (KEY_REVOKE)	Bit eFuse che invalida uno dei digest chiave; usato nella rotazione.
Rollback	Ritorno automatico allo slot precedente se la nuova applicazione non conferma il proprio corretto funzionamento.
RSA-3072 PSS	Algoritmo di firma usato da Secure Boot V2 (chiave a 3072 bit, padding PSS).
Secure Boot V2	Schema Espressif di verifica firma del bootloader e dell'applicazione basato su RSA/ECDSA e digest in eFuse.
secure_version	Numero monotono di versione di sicurezza nell'app descriptor, confrontato con l'eFuse per l'anti-rollback.

Termine	Definizione
Signature block	Struttura di 1216 byte appesa in coda a un'immagine, con chiave pubblica, firma e CRC. Fino a 3 per immagine.
Slot OTA (ota_0, ota_1)	Le due partizioni applicative alternate negli aggiornamenti: una in esecuzione, l'altra destinazione dell'update.
SUM	Secure Update Mechanism, termine usato nelle EN 18031 per il meccanismo di aggiornamento sicuro.
TLS	Transport Layer Security: protocollo che autentica il server (e opzionalmente il client) e cifra il canale.

11. Riferimenti

I link seguenti sono quelli che conosco come punti di ingresso della documentazione; non ho potuto verificarli online al momento della stesura. Se un link non risponde, cercare il titolo nella documentazione ESP-IDF della versione in uso.

11.1 Documentazione Espressif (ESP-IDF)

- Secure Boot V2 —
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/security/secure-boot-v2.html>
(selezionare il chip target nel menu della pagina)
- Flash Encryption —
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/security/flash-encryption.html>
- Security overview —
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/security/security.html>
- Over The Air Updates (OTA), rollback, anti-rollback —
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html>
- esp_https_ota — https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/esp_https_ota.html
- Partition tables —
<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/partition-tables.html>
- esptool / espsecure.py / espfuse.py — <https://docs.espressif.com/projects/esptool/en/latest/>
- Esempi: `examples/system/ota/` e `examples/security/` nel repository ESP-IDF

11.2 Normativa e standard

- Reg. (UE) 2024/2847 (Cyber Resilience Act) —
<https://eur-lex.europa.eu/eli/reg/2024/2847/oj/eng> — Allegato I (requisiti essenziali), Allegato VII (documentazione tecnica)
- Reg. delegato (UE) 2022/30 (RED Delegated Act) —
<https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022R0030>
- EN 18031-1/-2/-3:2024 — norme armonizzate per il RED-DA (disponibili tramite UNI/CEN)
- IETF RFC 9019 — A Firmware Update Architecture for Internet of Things (requisiti architetturali, modello SUIT)
- OWASP Cheat Sheet Series — <https://cheatsheetseries.owasp.org/> (TLS, Key Management, Secrets Management)
- NIST SP 800-193 — Platform Firmware Resiliency Guidelines (protezione, rilevamento, ripristino del firmware)

11.3 Prossimi passi suggeriti

Un percorso pratico per consolidare quanto letto, in ordine crescente di rischio:

- Abilitare la sola verifica di firma sugli update (sez. 5.7) su una scheda di sviluppo e osservare il rifiuto di un'immagine non firmata.
- Abilitare Secure Boot V2 in modalità development su una seconda scheda, con chiave dev, e ripetere i test negativi della sez. 4.5.

- Implementare rollback e anti-rollback nell'applicazione di esempio OTA di ESP-IDF; provocare un crash post-update e verificare il ritorno allo slot precedente.
- Solo dopo, su schede sacrificabili, provare una procedura completa in modalità release con Flash Encryption, documentandola come bozza della procedura di fabbrica.
- In parallelo, aprire la discussione organizzativa su HSM, firma remota e chiavi di riserva: è la parte che richiede più tempo di calendario.